

Universidad Nacional del Sur

Departamento de Ciencias e Ingeniería de la Computación

Informe Técnico de Arquitectura

Sílabus — UNS

Sistema de Gestión de Programas

v1.1.3-Beta

Masong Santiago

Junio 2026

1. Visión General del Sistema

Sílabus-UNS es una aplicación web para la Universidad Nacional del Sur que digitaliza, estandariza y centraliza el proceso de elaboración, revisión y aprobación de los programas de materias de todos sus departamentos.

El sistema sigue una arquitectura de 3 capas con separación estricta entre las tres desplegadas de forma independiente: una capa de presentación (Frontend), una capa de lógica de negocio (Backend) y una capa de persistencia (Base de Datos).

Componente	Tecnología	Versión	Despliegue
Frontend	Next.js + React + TypeScript	Next.js 16 / React 19	Vercel (CD automático desde rama main)
Backend	Spring Boot + Java 21	Spring Boot 3.5.6 / v0.9.0	Render (JAR empaquetado en imagen Docker)
Base de datos	PostgreSQL	Servicio administrado externo	Aivencloud (host en al nube independiente del backend)

1.1 Diagrama de arquitectura general

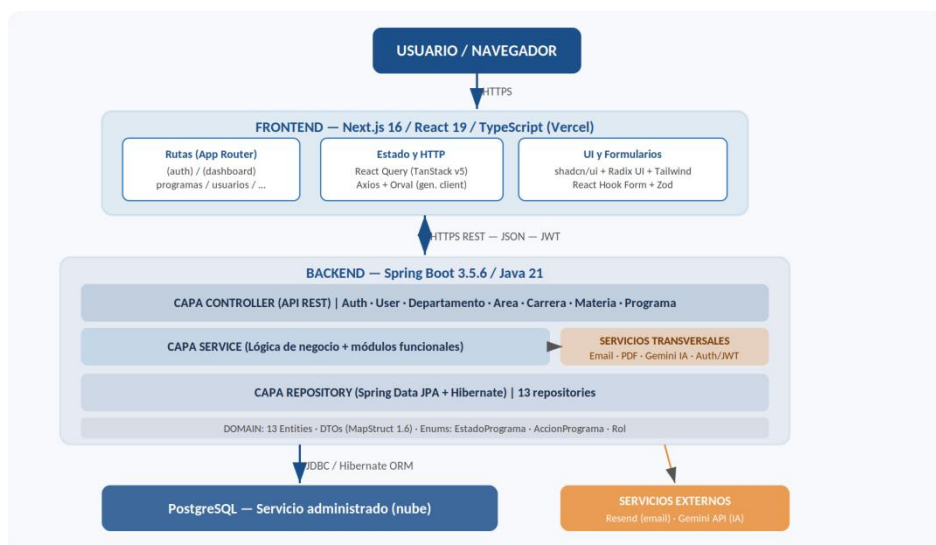


Figura 1: Vista completa del sistema — usuario, frontend (Vercel), backend (Spring Boot en Docker), base de datos PostgreSQL y servicios externos.

2. Arquitectura del Backend — Organización en Capas y Módulos

El backend es un monolito modular: toda la aplicación corre en una única instancia Spring Boot, pero el código está organizado en módulos funcionales bien delimitados. Sobre esos módulos se aplica una arquitectura en capas, de modo que cada módulo funcional posee sus propias clases en cada capa.

Dos ejes ortogonales: las CAPAS definen qué responsabilidad tiene el código (controller / service / repository); los MÓDULOS definen sobre qué parte del dominio trabaja (programas / usuarios / carreras...). No son alternativas sino dimensiones complementarias.

2.1 Las tres capas

Capa	Paquete Java	Responsabilidad
Controller	controller/	Recibe requests HTTP, valida entrada con Bean Validation, delega al Service y retorna la respuesta HTTP. No contiene lógica de negocio.
Service	services/ + services/impl/	Implementa las reglas de negocio: circuito de estados del programa, validaciones de rol y departamento, orquestación entre repositorios y servicios transversales.
Repository	repositories/	Acceso a datos mediante Spring Data JPA + Hibernate. Varios repositorios, uno por entidad principal.

2.2 Dominio

El módulo de dominio es el núcleo del sistema: contiene las entidades JPA, los Data Transfer Objects (DTOs) de transferencia, los enumerados y los mappers (MapStruct).

Su objetivo es representar las entidades del dominio, los efectos sobre ellas y transferir información entre capas mediante DTOs. Además se tiene el dominio sobre el cual actúa, la capa Service lo consume para ejecutar reglas de negocio, la capa Repository lo hidrata con datos de la base de datos, y la capa Controller lo referencia únicamente a través de DTOs.

Módulo de dominio	Paquete Java	Responsabilidad
Domain	domain/entities, domain/dto, domain/enums, mappers/	Entidades JPA, DTOs de transferencia, enumerados y mappers (MapStruct 1.6).

2.3 Diagrama de capas y sus relaciones

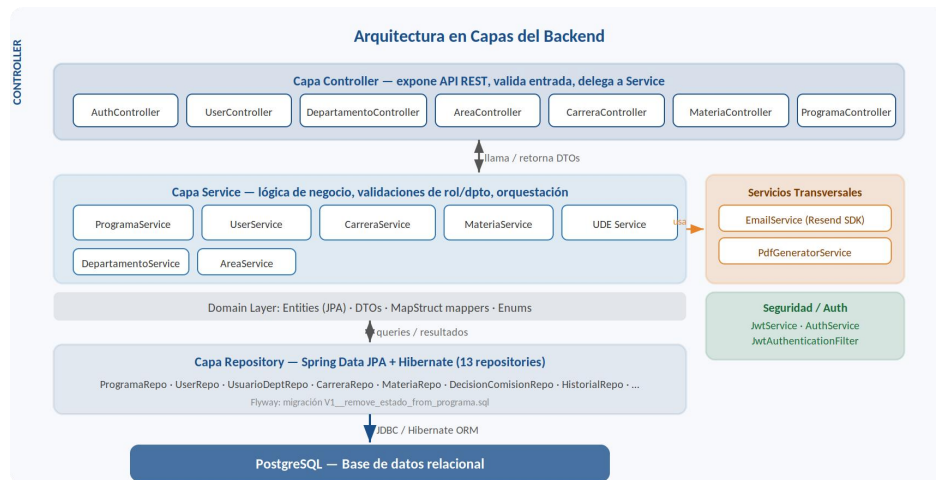


Figura 2: Organización en capas del backend. Las flechas bidireccionales entre Controller↔Service y Service↔Repository muestran llamadas y retorno de datos. Los servicios transversales y de seguridad se relacionan horizontalmente con la capa Service.

2.4 Módulos funcionales por capa

Cada módulo funcional del dominio tiene sus clases distribuidas en las tres capas:

Módulo	Controller	Service (impl)	Repositories principales
Programas	ProgramaController	ProgramaServiceImpl	ProgramaRepository, ProgramaCarreraRepo, DecisionComisionRepo, HistorialRepo, ProgramaDraftRepo
Usuarios	UserController	UserServiceImpl UsuarioDepartamentoServiceImpl	UserRepository, UsuarioDepartamentoRepository
Departamentos	DepartamentoController	DepartamentoServiceImpl	DepartamentoRepository
Áreas	AreaController	AreaServiceImpl	AreaRepository
Carreras	CarreraController	CarreraServiceImpl	CarreraRepository, CarreraPlanRepository
Materias	MateriaController	MateriaServiceImpl	MateriaRepository

2.5 Módulos transversales

Los módulos transversales no pertenecen a ningún módulo funcional en particular: proveen capacidades técnicas (autenticación, email, generación de PDF, llamada a IA) que pueden ser requeridas por cualquier módulo funcional. A diferencia de los módulos funcionales, no tienen una entidad de dominio propia y no siguen el esquema Controller–Service–Repository completo.

Módulo transversal	Paquete	Tecnología	Lo usan	Descripción
AuthController + AuthService + JwtService	controller/auth/ services/auth/	JWT 0.12.5 · Spring Security · BCrypt	JwtAuthenticationFilter (cada request)	Genera y valida tokens JWT (access 24h / refresh 7d). Expone AuthController para login y refresh. JwtAuthenticationFilter intercepta cada request antes del Controller para verificar el token y cargar el SecurityContext. No representa una entidad de dominio: es infraestructura de seguridad transversal.
EmailService	services/email/ /	Resend SDK · @Async · pool de threads configurable	ProgramaService, UserService	Envía 5 tipos de emails: bienvenida, notificación de carga administrativa, notificación docente, aprobación y rechazo con justificación. Ejecuta de forma asíncrona para no bloquear el request.
PdfGeneratorService	services/pdf/	Flying Saucer 9.1.22 + Thymeleaf	ProgramaService (/{id}/pdf)	Renderiza la plantilla Thymeleaf con los datos del programa aprobado, convierte el HTML resultante a PDF con Flying Saucer y retorna los bytes para descarga directa.
GeminiService	services/gemini/	Google Gemini 2.5 Flash · Spring WebFlux WebClient	ProgramaService(/form atear-apa)	Recibe texto bibliográfico libre, construye un prompt y llama a la API de Gemini para formatearlo en normas APA 7ma edición. Implementado con WebClient reactivo.

3. Módulo Programas

El módulo Programas es el núcleo funcional del sistema. Gestiona el ciclo de vida completo de un programa académico desde su creación hasta su aprobación final, coordina las decisiones multi-actor y multi-carrera, y es el principal consumidor de los módulos transversales (email, PDF, Gemini).

3.1 Entidades del módulo

El módulo agrupa varias entidades JPA que colaboran para modelar el proceso de aprobación:

Entidad	Responsabilidad
ProgramaEntity	Entidad central. Almacena todos los contenidos del programa (objetivos, bibliografía, cronograma, etc.), su estado actual (EstadoPrograma) y las referencias a la materia y al docente responsable.
ProgramaDraftEntity	Borrador temporal asociado a un programa. Permite edición incremental antes del envío formal, sin alterar el estado del programa publicado.
EstadoHistoricoEntity	Registro inmutable de cada transición de estado: actor, rol, fecha y justificación opcional. Provee trazabilidad completa del ciclo de revisión.
MateriaEntity	Materia fundamental a la cual el programa pertenece, la materia está relacionada y proporciona también el área y el departamento correspondientes.
ProgramaCarreraEntity	Relación N:M entre un programa y las carreras que lo incluyen. Cada combinación puede tener su propia DecisionComisionEntity.
DecisionComisionEntity	Relacionado mediante ProgramaCarreraEntity. Registra la decisión (aprobación o rechazo con justificación) de cada coordinador en cada carrera asociada al programa. La regla multicarrera: el estado solo avanza a APROBADO_POR_COMISION cuando todas las decisiones están aprobadas.
CarreraPlanEntity	Relacionado mediante ProgramaCarreraEntity. Indica en el plan y la carrera (mediante relación) del bloque. De esta manera un programa puede pertenecer a varios planes de varias carreras.

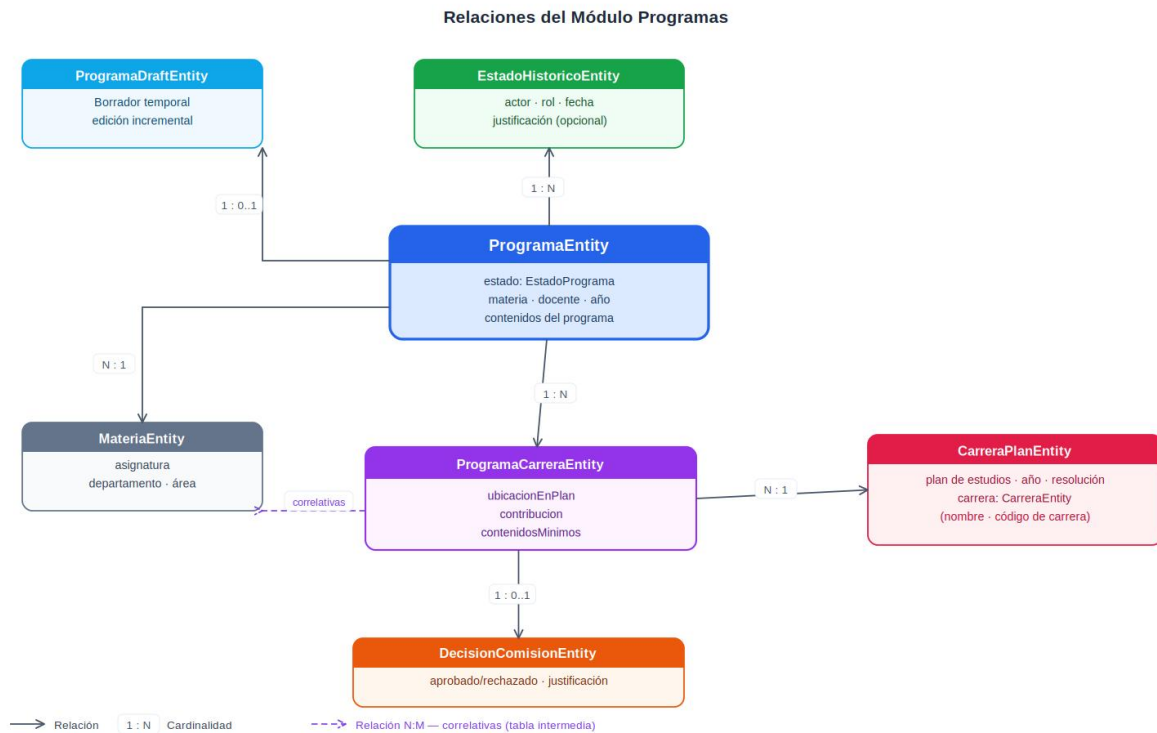


Figura 3: Diagrama de clases simplificado centrado en *ProgramaEntity* y sus relaciones más importantes dentro del sistema.

3.2 Ciclo de vida

El flujo de aprobación está modelado mediante el enum `EstadoPrograma` y ejecutado en `ProgramaServiceImpl`. Cada transición queda registrada en un `EstadoHistoricoEntity` con actor, rol, fecha y justificación en caso de rechazo.



Figura 4: Ciclo de vida completo de un programa.

Estado	Código enum	Actor	Transición siguiente
Completado por administración	COMPLETO_POR_ADMINISTRACION	Administrativo	Docente responsable es notificado y puede completar contenidos
Completado por docente	COMPLETO_POR_PROFESOR	Docente	Cada comisión curricular involucrada es notificada y puede aprobar o rechaza.
Rechazado a administración	RECHAZADO_A_ADMINISTRACION	Docente / Comisión / Secretaría	Administrativo es notificado y puede corregir y recargar
Rechazado a docente	RECHAZADO_A_PROFESOR	Comisión / Secretaría	Docente es notificado y puede corregir y recargar
Aprobado por comisión	APROBADO_POR_COMISION	Todas las comisiones	Secretaría Académica es notificada para revisar y puede aprobar o rechazar
Aprobado por secretaría ✓	APROBADO_POR_SECRETARIA	Secretaría Académica	Programa vigente, PDF disponible

Regla multicarrera: *DecisionComisionEntity* registra la decisión de cada coordinador por separado. Solo cuando todas las decisiones están aprobadas el estado avanza a APROBADO_POR_COMISION. El primer rechazo detiene el circuito.

3.3 Representación de estados en el frontend

Los valores del enum se persisten en el backend indicado el verdadero estado del programa y se exponen en la API tal cual. El frontend mantiene un mapa de traducción en `lib/utils` que convierte cada valor técnico a un nombre más amigable para el usuario final.

Valor técnico — enum EstadoPrograma (backend)	Etiqueta visible (frontend)
COMPLETO_POR_ADMINISTRACION	Pendiente Docente
COMPLETO_POR_PROFESOR	En Revisión (Comisiones Curriculares)
APROBADO_POR_COMISION	En Revisión (Secretaría)
APROBADO_POR_SECRETARIA	Aprobado
RECHAZADO_A_ADMINISTRACION	Rechazado a Administración
RECHAZADO_A_PROFESOR	Rechazado a Docente

4. Frontend — Módulos

El frontend está construido con Next.js 16 usando el App Router, organizado en dos grupos de rutas: las de autenticación (auth) y las de aplicación (dashboard) con el usuario ya autenticado con acceso al resto del sistema.

Módulo	Rutas	Definición
Autenticación	/app/(auth)/login /app/(auth)/forgot-password /app/(auth)/set-password	Rutas de autenticación
Aplicación	/app/(dashboard)/*	Rutas de aplicación del sistema una vez autenticado.

El módulo de aplicación está dividido en submódulos que se centran en la gestión de cada entidad dentro del sistema, siendo el submódulo de más importancia el de programas:

Submódulo	Rutas	Definición
Gestión de programas	/programas /programas/crear /programas/[id] /programas/[id]/carga/administracion /programas/[id]/carga/docente /programas/[id]/revision/comision/[carreraId] /programas/[id]/revision/secretaria /programas/[id]/historial-estados /programas/materia/[id]/versiones	Contiene las páginas y formularios para la visualización y para todas las etapas del ciclo de vida. Ciclo de vida: carga, revisión, corrección. Visualización: información, versiones por materia e historial de estados.
Gestión de departamentos	/departamentos	Visualización y CRUD de departamentos.
Gestión de áreas	/areas	Visualización y CRUD de áreas.
Gestión de carreras	/carreras	Visualización y CRUD de carreras.
Gestión de materias	/materias	Visualización y CRUD de materias.
Gestión de usuarios	/usuarios	Visualización y CRUD de usuarios.

Por supuesto también hay varios paquetes utilizados por los distintos módulos en todo el frontend como contextos, hooks, utilidades, componentes ui, etc:

Paquete	Rutas	Componentes	Utilizado por
Hooks	/hooks	Hooks: use-login,	Autenticación

Paquete	Rutas	Componentes	Utilizado por
		use-logout, use-forgot-password, use-set-password	
Contextos globales	/context /provider	AuthContext: usuario autenticado DeptContext: departamento activo seleccionado RolContext: rol activo dentro del departamento	Aplicación vía providers en el layout de la app
Componentes	/components/ui /components/forms /components/dashboards /components/modals /components/nav	Tiene componentes de ui, los formularios, dashboards, barras de navegación y auxiliares como popups, modals y toasts.	Aplicación
Utilidades y schemas	/lib/utils /lib/schemas	Helpers de formato, fechas, etc. Schemas Zod	Aplicación, Autenticación

5. Comunicación Frontend-Backend y autenticación

Toda la comunicación con el backend del módulo de aplicación es mediante clientes HTTP generados automáticamente por Orval a partir de la especificación OpenAPI del backend.

Con respecto al módulo de autenticación, este no se comunica con los clientes generados por Orval, ya que primero se debe autenticar al usuario para poder usarlos, sino que se comunica directamente con los endpoints del backend.

5.1 Flujo de comunicación Frontend → Backend mediante Orval

Como se mencionó la comunicación para el módulo de aplicación (cuando ya se está autenticado) es mediante Orval. El mismo es una biblioteca de gestión de API que genera automáticamente clientes tipados (en TypeScript) de manera segura a partir de cualquier especificación OpenAPI o Swagger del backend.

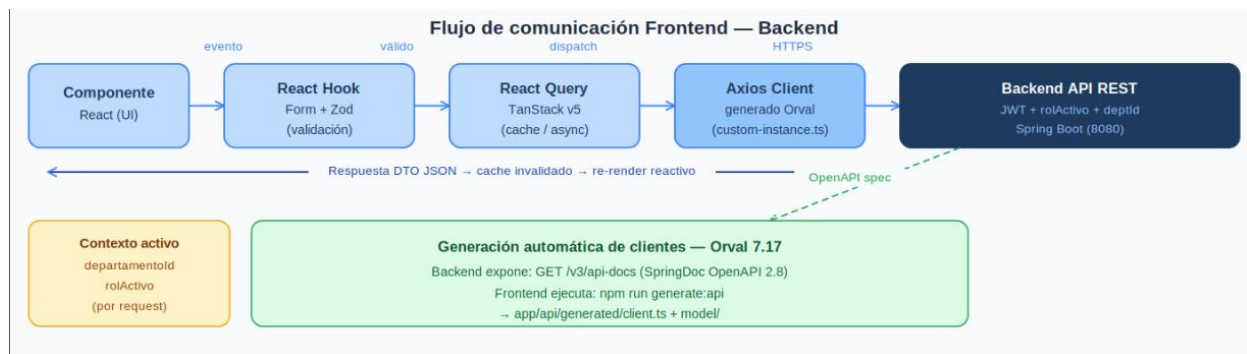


Figura 5: Pipeline de comunicación autenticada enfocado en frontend. Un evento del usuario atraviesa validación (React Hook Form + Zod), gestión de estado asíncrono (React Query), el cliente HTTP generado por Orval, y llega al backend con JWT y contexto activo. La respuesta actualiza el cache y re-renderiza la UI.

5.2 Proxy Next.js y arquitectura de autenticación

La comunicación entre el frontend y el backend no es directa desde el browser: el servidor de Next.js actúa como mediador en dos roles complementarios.

Login (Módulo Autenticación): el browser hace un fetch a `/api/auth/login` que es un Route Handler server-side de Next.js. Éste llama al backend Spring Boot, recibe el JWT y lo persiste como cookie `httpOnly` en el mismo origen (Vercel). El JavaScript del browser nunca accede al token.

Requests autenticadas (Módulo Aplicación): el `customInstance` de Orval apunta al server-side de next.js donde se redirigen esas llamadas al backend como una request server-to-server, adjuntando automáticamente la cookie. El browser siempre habla con su mismo origen (Vercel) y nunca con el backend directamente.

Justificación: la comunicación directa browser → backend era inviable en producción. Vercel (frontend) y Render (backend) son dominios distintos: las restricciones CORS del browser y la política `SameSite=None` impedían intercambiar cookies `httpOnly` entre orígenes distintos. El servidor de Next.js resuelve esto actuando como mediador: todas las operaciones sensibles ocurren server-side, donde CORS no existe.

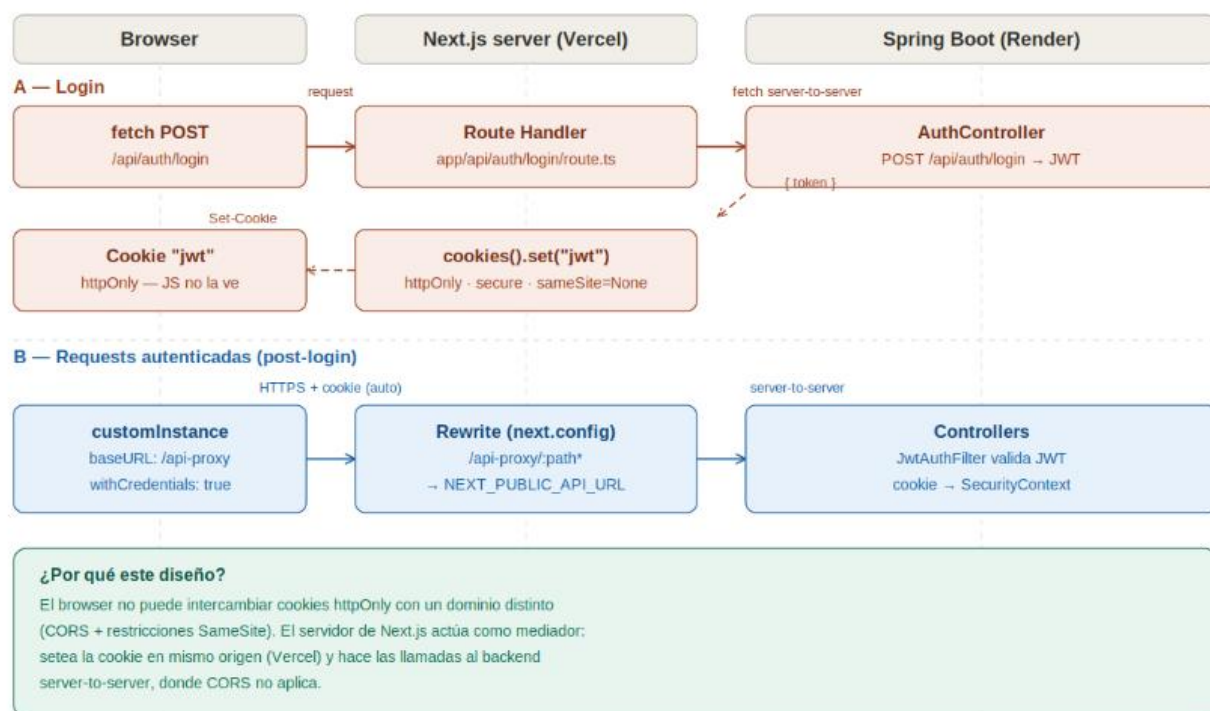


Figura 6: Flujo A (login vía Route Handler, Módulo Auth) y Flujo B (requests autenticadas vía rewrite proxy, Módulo App). Las llamadas server-to-server no están sujetas a restricciones CORS del browser.

6. Autenticación y seguridad contextual

6.1 Modelo de Seguridad Contextual

El aspecto más complejo y más importante del sistema desde el punto de vista arquitectónico es la gestión multicontextual de usuarios. Un mismo usuario puede pertenecer a múltiples departamentos y tener roles distintos en cada uno. Esto impacta directamente en la capa Service, en el modelo de datos y en cada validación de seguridad que se lleva a cabo por cada request que hace el usuario.

UsuarioDepartamentoEntity es la entidad pivote del modelo de seguridad, manejada por el módulo de Usuario. Representa la pertenencia de un usuario a un departamento y almacena: conjunto de roles en ese contexto, email institucional departamental, historial de acciones, lista de materias asignadas (si es docente), lista de carreras coordinadas (si es coordinador).

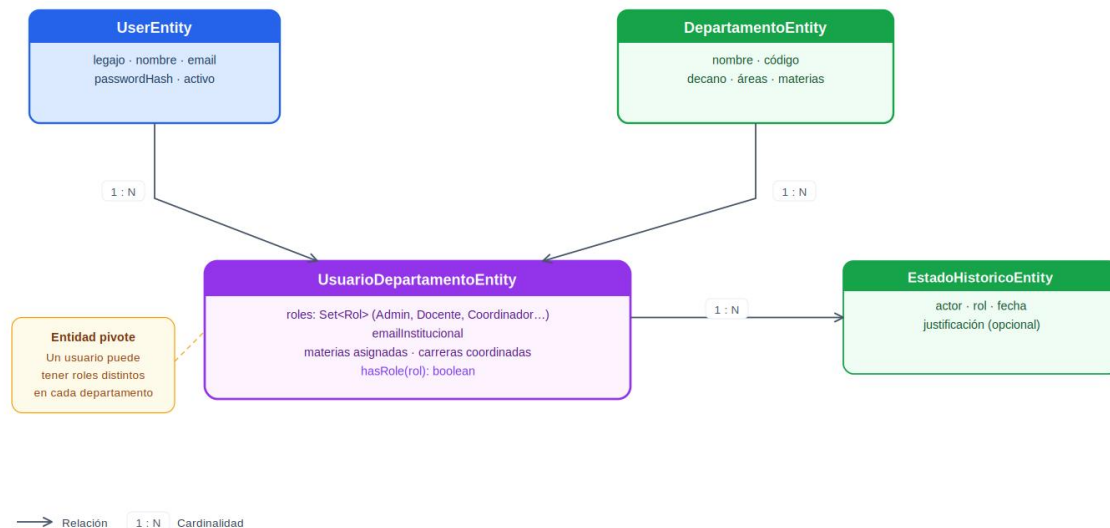


Figura 7: Diagrama de clases simplificado centrado en UsuarioDepartamentoEntity y sus relaciones más importantes dentro del sistema.

6.2 Flujo de autenticación, selección y validación de contexto

Paso 1 — Login:

POST /api/auth/login → Spring Security valida credenciales → JwtService genera accessToken (24h) y refreshToken (7d).

Paso 2 — Selección de departamento:

El frontend muestra los departamentos disponibles para el usuario y pide que seleccione uno. El departamentoId queda almacenado en el contexto del frontend.

Paso 3 — Selección de rol:

Dentro del departamento, el usuario elige un rol de los que tiene asignados en esa UsuarioDepartamentoEntity. El rolActivo queda en el contexto del frontend.

Paso 4 — Cada request:

El frontend adjunta JWT en el header Authorization: Bearer ... y envía departamentold + rolActivo como parámetros de cada llamada API.

Paso 5 — Validación en Service del backend:

Cada método de Service llama a `findByUsuarioLegajoAndDepartamentoId()` y verifica si el usuario tiene ese rol en el departamento con el método `hasRole(rolActivo)` antes de ejecutar cualquier operación. Si falla → `AccessDeniedException`.

El rol activo del usuario no pertenece a la lista de roles del usuario en ese departamento y por ende no tiene permisos.

Además algunas operaciones y métodos retornan distinta información dependiendo del rolActivo del usuario en ese departamento.

7. Despliegue e Infraestructura

El sistema utiliza Docker para empaquetar y ejecutar el backend, y Vercel para desplegar el frontend con CI/CD automático.

Servicio docker-compose	Imagen / Build	Puerto	Variables de entorno requeridas
backend	Dockerfile en Backend/ (FAT JAR Spring Boot)	8080:8080	SPRING_DATASOURCE_URL, _USERNAME, _PASSWORD · JWT_SECRET · SERVER_PORT · RESEND_API_KEY · RESEND_FROM_EMAIL · GEMINI_API_KEY
frontend	Dockerfile en Frontend/ (Next.js)	3000:3000	NEXT_PUBLIC_API_URL

La base de datos PostgreSQL no corre en Docker local sino en un servicio externo de Aivencloud. El backend dockerizado se despliega en Render y se conecta mediante JDBC con las credenciales configuradas en variables de entorno. El frontend en producción se despliega en Vercel desde el branch main con CI/CD automático.